In [17]:
```
import random
import numpy as np
```

In [18]:
```
max_int = 7 #entries are random integers from -max_int to max_int
N = 6 #size of system: N equations, N unknowns
A = np.zeros((N,N)) #coefficient matrix
b = np.zeros((N,1)) #right-hand side as a column vector

# Fill A and b with random integers
for i in range(N):
    for j in range(N):
        A[i][j]=random.randint(-max_int,max_int)
        b[i][0]=random.randint(-max_int,max_int) #for every j, overwrite
b[i], and then keep the last value

# Create a singular matrix
# Row 2 becomes minus row 1, so the rows are linearly dependent and we have
det(A) = 0
A[[2]]=-A[[1]]
b[2]=-b[1] #system has infinite solutions

print(np.matrix(A))
print(np.matrix(b))
```

```
[[-4.  7.  3. -2.  6.  7.]
 [ 7. -7.  5.  6.  3.  7.]
 [-7.  7. -5. -6. -3. -7.]
 [-2. -5.  4.  6.  1. -3.]
 [ 5. -4. -5. -7.  7. -2.]
 [ 4.  4.  2. -1. -7.  2.]]
[[-2.]
 [-6.]
 [ 6.]
 [ 5.]
 [-6.]
 [-5.]]
```

I check if A is singular with Gaussian elimination with partial pivoting. If at a column the largest value I can pick as pivot is 0, the rows are linearly dependent and the matrix is singular.

In [19]:
```
def singular_test(A):
    U = np.array(A, dtype=float) #copy
    n = U.shape[0] #number of rows
    # Rounding errors a pivot is almost never 0 so I call it 0 when it is
very small compared to the entries of A
    t = 1e-10 * np.max(np.abs(U))

    # Go through the columns one by one, like in Gaussian elimination
    for j in range(n):
        # Partial pivoting: row with the largest |U[i, j]| from row j down
        p = j + np.argmax(np.abs(U[j:, j])) #argmax counts from row j, so I
add j to get the actual row number
        U[[j, p]] = U[[p, j]] #swap row j and row p

        # largest value in the column is 0 → this column has no pivot →
rows are linearly dependent and matrix is singular
        if np.abs(U[j, j]) ⩽ t:
            print("Matrix is singular. This system has no unique solution.")
            return True

        # Make the elements below the pivot 0, this does not change the
determinant
        for i in range(j + 1, n):
            m = U[i, j] / U[j, j] #multiplier, chosen so that U[i, j]
becomes 0
             # row i = row i - m * row j
            U[i, j:] -= m * U[j, j:]
    print("Matrix is nonsingular. This system has a unique solution.")
    return False
```

In [20]:
```
singular = singular_test(A)
```

```
Matrix is singular. This system has no unique solution.
```